

Fee Collection SaaS for Private African Schools

A resilient, multi-tenant platform built for the financial realities of Central & West Africa: integer minor-unit ledgering, Postgres Row-Level Security, multi-channel automated reminders, and real-time reconciliation.

ARCHITECTURE

NestJS + Postgres RLS
+ Redis

FINANCIAL MODEL

Append-Only Double
Entry

CURRENCY SUPPORT

0-decimal (XAF, XOF)
+ ISO

Table of Contents

01 Executive Summary & Product Requirements (PRD)

02 Architecture Essentials (Quick Reference)

03 Complete Technical Architecture & Specifications

04 Development Principles & Invariants (AGENTS.md)

Executive Summary & Product Requirements (PRD)

PRD — Fineduc

Product Requirement Document. What we are building, who it is for, and what the product actually needs to do. Product-level only — no technology decisions. Technology lives in [ARCHITECTURE.md](#).

- **Status:** v1 draft, pre-code
- **Owner:** founder
- **Last updated:** 2026-08-18

1. Problem

Private schools in francophone and anglophone Africa run on a broken fee-collection loop.

Symptom	What it costs the school
Chasing fees by hand — the bursar calls parents one by one, class list in hand	Weeks of staff time per term; collection depends on one person's memory
Late payments — no schedule the parent can see, no reminder before the due date	Cash arrives after salaries and suppliers are due; the school borrows to bridge
Untracked cash — parents pay in banknotes at the desk, receipt book is a carbon pad	Money leaks between the desk and the safe; nobody can prove what was collected on a given day
Millions never recovered — arrears roll from term to term, then year to year	At year-end the school writes off a large share of what it was owed
The director is blind — the real position is known only after a manual month-end reconciliation	Decisions (hiring, works, salaries) are made on a number that is weeks stale

The root cause is not fraud and not laziness. It is that **the fee ledger only exists on paper and in one person's head**, so nothing about it can be scheduled, reminded, reconciled, or seen in real time.

2. Solution

Fineduc gives every student a file, every fee a due date, and every payment a trace.

Four moves, in the order they unlock value:

1. **A file per student** (*un dossier par élève*) — one record holding the student, the guardians who pay, what is owed, what has been paid, what remains. Replaces the register, the receipt book and the bursar's memory.
2. **Instalments** (*des échéances*) — the year's fees split into dated tranches the moment the student is enrolled. The parent knows the amount and the date in advance.
3. **Automatic reminders** (*rappels automatiques SMS / WhatsApp*) — the system messages the guardian before the due date, on the due date, and after. Nobody has to remember.
4. **Mobile money payment** — the parent pays from the phone that received the reminder, through a **third-party payment aggregator** (never a direct telco integration). Reconciliation is automatic because the payment arrives already attached to a student.

And the outcome the buyer actually pays for:

5. **The director sees everything, in real time** (*le directeur voit tout en temps réel*) — collected today, expected this month, arrears by class, cash desk status, all live.

Positioning

Fineduc — the school fee ledger that collects by itself. *Un dossier par élève, des échéances claires, des rappels automatiques, le paiement mobile money. Le directeur voit tout en temps réel.*

3. Who it is for

Buyer / champion — **the Director / Proprietor** (*Directeur, Promoteur, Fondateur*)

Owns the P&L. Feels the pain as a cash-flow number. Buys because of the real-time view and the recovery rate.

Rarely opens a computer — decides from a phone. Success = "*I know my position without asking anyone.*"

Daily driver — **the Bursar / Accountant** (*Économe, Comptable*)

Lives in the product 4+ hours a day. Enrols students, builds fee schedules, allocates payments, closes the cash desk, chases arrears. Will kill the product if it is slower than their notebook. Success = "*I closed the desk in five minutes and it balanced.*"

Cashier (*Caissier*)

Takes cash and issues receipts at the desk. Often junior. Needs three taps, not thirty. Success = "*receipt printed, parent gone.*"

Payer — **the Guardian** (*Parent, Tuteur*)

Never logs in. Receives WhatsApp/SMS, taps a link, pays with mobile money, gets a receipt back on the same thread. Frequently: low literacy, shared phone, several children in the school, pays in irregular slices as income allows. Success = "*I paid without going to the school.*"

Not our user in v1

Teachers, students, ministry inspectors. No grades, no attendance, no timetable. **Fineduc is not a school management system.** It is a fee-recovery system. This boundary is the product.

4. Scope

v1 — must ship

Student file

- Student record: identity, photo, class, academic year, status (enrolled / left / graduated)
- Guardians attached to a student, each with a phone number, a relationship, and a `pays_fees` flag; one guardian may hold several students
- Enrolment (*inscription*) into a class for an academic year — the act that creates money owed
- Re-enrolment for the next year, carrying the outstanding balance forward
- Sibling linkage (same guardian → one consolidated view, one message)

Fees and instalments

- Fee schedule (*grille tarifaire*) per class per academic year, composed of fee items: tuition (*scolarité*), registration (*frais de dossier*), exam, canteen, transport, uniform, boarding — each recurring or one-off
- Instalment plan (*échancier*): the year's total split into dated tranches (typically three, configurable per school and per class)
- Discounts and scholarships: sibling, staff-child, merit, hardship — percentage or fixed amount, applied at fee-item or student level, and **always attributable to whoever granted it**
- Per-student payment arrangement (*arrangement*) overriding the class plan, with an approval trail

Payment

- **Mobile money via a third-party aggregator** — MTN MoMo, Orange Money, Moov, Wave, card, all reached through one aggregator account. Fineduc holds **no** direct telco contract and **no** licence.
- Cash at the desk, always inside a cash session (below)
- Bank transfer / cheque, recorded manually with a reference
- **Partial payment is the norm, not the exception.** A parent paying 12 000 against a 45 000 instalment must be a first-class, one-tap operation
- Allocation: a payment settles the oldest instalment first by default, with manual override
- Receipt on every payment — numbered, immutable, re-sendable to the guardian
- Overpayment becomes a credit balance carried to the next instalment
- Refund and reversal — restricted, reason-coded, dual-approval, never a delete

Cash control — the answer to *espèces mal tracées*

- Cash session: a named cashier opens a desk with a declared opening float, takes payments, closes with a declared count
- The system computes expected cash, shows the variance, and **requires a reason for any variance**
- A session cannot be reopened after close; corrections are new, signed entries
- Daily cash report per cashier, per desk, per day — the artefact the director reviews

Reminders

- Reminder rules per school: offsets relative to the instalment due date (D-7, D-2, D-day, D+3, D+10, D+30), each with a channel and a template
- Channels: **WhatsApp first** (cheap, rich, threaded), **SMS fallback** (universal, works on feature phones with no data)

- Templates in French and English, per school, with variables ({parent} , {eleve} , {classe} , {montant} , {echeance} , {lien}) and a school signature
- Every reminder carries a **payment link** — tap, pay, done
- Escalation ladder: gentle → firm → formal notice, tone set by the school
- Quiet hours, per-guardian frequency cap, immediate stop on payment
- Opt-out honoured and recorded
- Full message log: sent, delivered, read, failed, cost — visible to the bursar
- Manual send: to one guardian, one class, or a filtered debtor list

Director dashboard, real time

- Collected today / this week / this term, against expected
- Recovery rate for the term and for the year
- Arrears (*impayés*) ranked by class, by amount, by age bucket (0-30 / 31-60 / 61-90 / 90+ days)
- Cash desk status right now: open sessions, cashier, amount held
- Payment-method mix (mobile money vs cash vs transfer) — the number that proves cash is shrinking
- Reminder performance: sent, delivered, and amount collected within 72h of a reminder
- **Mobile-first.** The director looks at this on a phone, standing up. Design for that first.
- Export to Excel/PDF for the board, the auditor, the bank

Administration and security

- Multi-tenant: one school = one tenant, hard-isolated. A group with several campuses is one tenant with several sites.
- Roles: Director, Bursar, Cashier, Secretary, Read-only (auditor). Least privilege by default.
- Two-factor for Director and Bursar
- Append-only audit log on every money-touching and permission-touching action — who, what, before, after, when, from where
- Academic year lifecycle: open → active → closed. A closed year is read-only.
- Full data export owned by the school

v1.1 — next, not now

Parent self-service portal · standing orders / auto-debit · bulk enrolment import from Excel · offline-capable cashier mode · payment-plan simulator · WhatsApp inbound (parent replies "solde", gets their balance) · USSD/IVR balance check.

Explicitly out of scope

Grades, attendance, timetable, payroll, library, transport routing, HR, e-learning, a general ledger, direct telco payment integration, and holding funds. **Fineduc is never a payment institution** — money moves aggregator → school account, never through us.

5. Constraints that shape everything

These are not footnotes. They are the design.

- **The currency has no decimals.** XAF and XOF are ISO-4217 exponent **0**. There are no centimes. Every money value is an integer of the currency itself. Any float, any `amount * 100`, any two-decimal display is a bug.
- **The network is bad.** 2G/3G, dropped requests, duplicate submissions. Every write that moves money must be idempotent. The cashier *will* press the button twice.
- **Phone numbers are the identity.** Guardians are reached by phone, not email. Numbers are shared between households, change often, and are entered inconsistently (`+237 6xx` , `06xx` , `2376xx`). Normalise to E.164 on the way in, always.
- **Messaging costs real money.** Every SMS is a cost line; WhatsApp utility conversations are far cheaper. Reminder volume must be capped, budgeted and visible — a runaway loop is a financial incident, not just a bug.
- **Payment confirmation is asynchronous and unreliable.** The aggregator's webhook may arrive late, twice, or never. Never trust the browser redirect. The webhook is authoritative; polling reconciliation is the safety net.
- **Two languages, one product.** French-first UI, English available. Templates in both.
- **Low-end Android, small data budget.** The dashboard must be usable on a three-year-old phone over 3G.
- **Trust is the sale.** These schools have been burnt by software vendors and by staff. The audit trail and the cash reconciliation are not compliance features — they are the demo.

6. What success looks like

Metric	Target by end of first full term
Collection rate at term end	+15 points vs the school's previous term
Share of fees paid by mobile money	> 40% (from ~0)
Cash desk variance	< 0.5% of cash collected, every variance reasoned
Bursar time spent chasing	–60% self-reported
Amount collected within 72h of a reminder	tracked, reported, used in the sales pitch
Director dashboard opened	≥ 4 days a week
Reminder delivery rate	> 95%

Anti-metric, watched weekly: **messaging cost per 1 000 XAF collected**. If it rises, the reminder ladder is wrong.

7. Pricing

Priced off what a school actually recovers, and off what it costs us to serve them.

Cost floor per school per month

- Infrastructure (shared): ~2 000–4 000 XAF
- WhatsApp utility conversations: ~3–15 XAF each
- SMS: ~15–25 XAF each — **the dominant variable cost**

- Payment aggregator: 1.5–3.5% of value, **paid by the payer or the school, never absorbed by us**
- Support and onboarding, amortised: ~10 000–20 000 XAF

A 300-student school on a three-reminder ladder generates ~900 messages/month. All-SMS that is ~18 000 XAF. **This is why messaging is a metered wallet, not an unlimited inclusion.**

Model: platform subscription (sized by enrolled students) + prepaid message wallet

Plan	For	Price	Included
Essentiel	up to 250 students	25 000 XAF / month (~\$40)	Student files, instalments, cash sessions, mobile money, director dashboard, 2 staff + 2 cashiers, 500 WhatsApp msgs/mo
Croissance	251–800 students	60 000 XAF / month (~\$100)	Everything, unlimited users, escalation ladder, multi-desk cash control, Excel/PDF export, 2 000 WhatsApp msgs/mo , priority support
Institution	800+ / multi-campus	from 120 000 XAF / month	Everything, multi-site consolidation, custom templates, API access, named account manager, on-site onboarding

Annual, paid up front: –20%. This matches how schools budget — they pay once, at the start of the year, out of registration income. Expect most revenue here.

Message wallet — prepaid credits, never a surprise bill:

- WhatsApp **10 XAF** / message · SMS **30 XAF** / message
- Packs: 10 000 msgs for 85 000 XAF · 50 000 msgs for 375 000 XAF
- The school sets a monthly cap. At the cap, sending stops and the bursar is warned.

One-off onboarding: 150 000 XAF — migration from the existing registers, fee-schedule setup, two staff training sessions. Waived on an annual Croissance or Institution contract. This is not a margin line; it is the thing that makes the school actually adopt.

Aggregator fees are passed through, configurable per school: charged to the parent (default) or borne by the school. Fineduc never touches the money and never nets a fee out of it.

Sanity check

A 400-student school charging 250 000 XAF/year collects 100 000 000 XAF. Croissance annual is 576 000 XAF — **0.58% of collections**. Recovering even 3% more arrears returns 3 000 000 XAF. The pitch is one sentence: *you pay us out of what we recover for you.*

Deliberately not doing

Percentage-of-collections pricing (feels like a tax, invites disputes, drags us toward payment-institution regulation) · a free tier (these are businesses; free attracts schools that never adopt) · per-user seats (punishes the cashier count, which we want high).

8. Landing page

Marketing site for African private schools. French-first, English toggle.

Design reference: the FintechX Framer template mirrored in `D:\mes site\fineduc`. Dark `#1d1d1d` on off-white `#edf1f4`, slate body `#4d585f`, accent blue `#3b82f6`, positive emerald `#10b981`, Inter throughout. Generous whitespace, large rounded cards, soft shadows, scroll-triggered reveals, sticky-scroll step stacks, a live-looking dashboard mock as the hero. Calm, financial, credible — **not** a colourful edtech look. These buyers need to see a bank, not a school.

Section order

1. **Hero** — "Arrêtez de courir après les frais de scolarité." Sub: one file per student, automatic reminders, mobile money payment, real-time view. CTAs **Demander une démo** / **Voir le tableau de bord**. Trust strip: schools using it · students managed · amount recovered.
2. **Avant / Après Fineduc** — the template's before/after toggle, carrying our five pains and five fixes.
3. **Core features** — student file, instalments, reminders, mobile money, cash control, director view.
4. **Director dashboard showcase** — the real screen, on a phone. This is the money shot.
5. **How it works**, three sticky steps — *Import your students* → *Set your fee schedule* → *Reminders and payments run by themselves*. "Live in 48 hours."
6. **Security and trust** — audit trail, encryption, the data belongs to the school, no funds held, role separation. Written to convince a suspicious director.
7. **Who it is for** — nursery/primary, secondary, bilingual, technical, multi-campus groups.
8. **Numbers** — recovery-rate lift, cash-share reduction, time saved. Real ones, once we have them.
9. **Testimonial** — one director, named, with a number.
10. **Pricing** — three plans, annual toggle, wallet explained plainly.
11. **FAQ** — Do you touch our money? (no) · Which mobile money operators? · Does it work when the internet is down at the desk? · Can parents pay in slices? · We already use Excel — why change? · Who owns the data?
12. **Final CTA + footer**.

Pages: `/` · `/fonctionnalites` · `/tarifs` · `/securite` · `/demo` · `/contact` · `/blog` · `/legal/*`. A WhatsApp contact button on every page — that is how these buyers get in touch.

9. Hard questions

Asked of the plan before writing code. The answers are kept here because they are requirements.

What will break?

- **Reconciliation drift.** The aggregator says paid, we say pending, the parent has a confirmation SMS and is standing at the desk. → The webhook is authoritative; a scheduled job re-queries every non-final payment; a manual "force reconcile" exists and is audited.
- **Double payment.** Bad network, parent taps twice, two debits. → Idempotency key on every initiation, keyed to student + instalment + amount + a short window; automatic detection of duplicate settlements and a one-tap refund path.
- **The reminder storm.** A bug, a retry loop, or a bad rule messages 3 000 guardians at 02:00. Simultaneously a money loss, a trust loss, and possibly the end of the contract. → Hard per-tenant daily cap, quiet hours enforced at the *send* layer (not the scheduler), per-guardian frequency cap, kill switch, and every send debited from a visible wallet.

- **Reminding someone who already paid.** The fastest way to lose a school. → Reminder state is resolved at send time against the live balance, never at schedule time.
- **A wrong phone number.** The number belongs to a stranger, or to a guardian who left — and now a stranger receives a debt notice naming a child. That is a privacy incident. → Number validation, failure tracking that quarantines a number after N bounces, and guardian confirmation before the first send.
- **A cash session never closed.** The cashier goes home. → Auto-flag at end of day, escalate to the bursar, block the next session on that desk.
- **Timezone and due-date arithmetic.** "Due today" computed in UTC fires reminders a day early. → Every tenant has a timezone; every due date is a calendar date in tenant time, never a timestamp.
- **Mid-year fee changes.** The school raises fees in January; some students already have instalments. → Fee schedules are versioned and effective-dated; existing instalments are never silently mutated — a change creates an adjustment line.
- **The bursar who does not trust it.** Runs the notebook in parallel for a term, then the two disagree. → Import their notebook on day one; make the daily cash report match their format.
- **Aggregator outage on the last day of term.** Peak day, provider down. → Cash and transfer paths never depend on the aggregator; queue and retry initiations; show status honestly to the bursar.

What edge cases are we missing?

Student leaves mid-year holding a credit balance · a guardian pays for three children in one transfer · two guardians pay the same instalment at the same moment · a payment arrives for a student who was never enrolled (wrong reference) · sibling discount when one sibling withdraws · the academic year is closed but a late payment arrives for it · a guardian phone shared by two families · a partial refund on a partially-allocated payment · a school changing its instalment count mid-year · a cashier who is also a guardian · a payment in a currency other than the tenant currency · a student enrolled in two schools of the same group.

What is over-engineered?

Cut from v1, deliberately: a parent-facing portal (**WhatsApp is the portal**) · a full double-entry general ledger (a student ledger plus a cash ledger is enough — we are not the accountant) · a rules engine for reminders (a fixed offset ladder covers every school we have spoken to) · offline-first sync (a slow online desk beats a wrong offline one — revisit in v1.1 with real data) · microservices (one deployable, clean modules) · multiple aggregators on day one (one, behind a port; a second when a real school demands it) · in-app chat, an analytics warehouse, ML arrears prediction, and a native mobile app (the dashboard is a good web app).

Architecture Essentials (Quick Reference)

ARCHITECTURE-ESSENTIALS — Fineduc

*The **outline only**. Load this on every task. Open the full [ARCHITECTURE.md](#) only when you need the detail behind a line here.*

What it is

Fee-collection SaaS for private schools in Africa. One file per student, dated instalments, automatic WhatsApp/SMS reminders, mobile-money payment through a **third-party aggregator**, real-time director dashboard. **Not** a school-management system.

Stack

TypeScript · Node 22 · **NestJS** API · **PostgreSQL 16** (+RLS) · **Prisma** · **Redis** · **BullMQ** workers · **Zod** contracts · **Next.js 15** dashboard / pay page / landing · S3 (R2) · Sentry + OTel. Monorepo: pnpm workspaces + Turborepo. Modular monolith — **no microservices, no GraphQL**.

Layout

```
apps/      api · worker · dashboard · pay · web
packages/ domain · db · contracts · providers · money · config · ui
```

`domain` imports nothing. Apps never import from apps. Enforced by lint in CI.

The ten rules — break one and the system is wrong

1. **XAF/XOF have ZERO decimals.** Money is `bigint` minor units + an explicit currency exponent. No floats. No hard-coded `× 100`. All arithmetic via `packages/money`.
2. **Money tables are append-only.** Never update, never delete a payment or a ledger entry. Corrections are new, signed, reason-coded rows.
3. **Balances are projections**, written only in the same transaction as the ledger entry that changes them, and verified by the nightly integrity sweep.
4. **Tenant isolation is Postgres RLS.** `set local app.tenant_id` inside a transaction, on every request and every job. The API role has **no** `BYPASSRLS`.

5. **Idempotency-Key** is mandatory on every money-moving POST and every message send.
6. **The webhook is authoritative; the browser redirect is a hint.** Store the raw event first (unique on `provider + event_id`), return 200 fast, do the work in a job.
7. **Reminder eligibility is decided at SEND time, not schedule time.** Re-check the live balance. Messaging a parent who already paid is the fastest way to lose a school.
8. **Provider names never appear in domain code.** Payment and messaging are ports with swappable adapters. No direct telco integration, ever.
9. **due_on** is a **DATE** in tenant timezone. Never a timestamp. Never UTC arithmetic.
10. **Every mutating request writes an `audit_log` row.** That table has INSERT/SELECT grants only.

Core data model

```
tenant - site - user - membership(role)
academic_year - term - grade_level - class_group
student - student_guardian - guardian
enrollment          ← the act that creates money owed
  ↳ invoice - invoice_line - discount - adjustment
    ↳ instalment ← what reminders and payments hang off
      ↳ payment_allocation - payment - receipt / refund
student_ledger_entry ← APPEND-ONLY truth of a student's account
cash_desk - cash_session - cash_movement ← the anti-leak control
reminder_rule - reminder_schedule - message - message_credit_ledger
message_template · payment_link · provider_event · outbox · audit_log
```

Modules

`identity` · `tenancy` · `academics` · `students` · `billing` · `payments` · `cashbox` · `messaging` · `analytics` · `audit` · `platform`. Cross-module calls go through the public service interface only — never into another module's repository.

The four flows that matter

- **Enrol** → one transaction: enrollment + invoice + lines + discounts + instalments + ledger entries. Invariant: `Σ instalment.amount == invoice.net_minor`.
- **Pay (mobile money)** → initiate (idempotent, re-validate against the **live** balance) → aggregator → **webhook** → job → lock invoice `FOR UPDATE` → allocate oldest-due-first → ledger → cancel that instalment's reminders → receipt.
- **Pay (cash)** → requires an open `cash_session`; same allocation path; **gapless** receipt number from a locked counter row (not a Postgres sequence — sequences gap on rollback).
- **Remind** → nightly scheduler materialises intent; the sender every 15 min re-checks: paid? opted out? quarantined? quiet hours? frequency cap? tenant daily cap? credits > 0? — **all limits live in the sender, not the scheduler.**

Security

argon2id · JWT 15 min + rotating refresh with reuse detection · TOTP 2FA for Director & Bursar · RBAC deny-by-default (Director / Bursar / Cashier / Secretary / Auditor) · **four-eyes** on refunds, reversals, large discounts and cash-variance reconciliation · Zod at every boundary · rate limits · signature-verified webhooks with a replay window · payment-link tokens carry no PII · PII redacted from logs · secrets from env only, validated at boot. **Fineduc never holds funds** — the aggregator settles directly to the school's account.

Providers (v1)

Payment: **CinetPay** primary, Flutterwave secondary, Manual (cash/transfer), Fake (tests). Messaging: **WhatsApp Cloud API** first (cheap), SMS aggregator fallback, Console/Fake. Every adapter must pass the same port contract test suite.

Jobs

webhook-processor · reminder-scheduler (02:00 tenant tz) · message-sender (15 min) · message-status · reconciler (30 min) · integrity-sweep (03:00) · receipt-renderer · exporter · director-digest (07:00 tenant tz) · outbox-publisher (5 s). Every payload carries `tenantId` + `requestId` and sets `app.tenant_id`.

API

REST `/api/v1`, Bearer JWT, OpenAPI from Zod. Cursor pagination. RFC 9457 problem+json errors. Money on the wire: `{ "amountMinor": "45000", "currency": "XAF" }` — **a string**, so no client loses precision. Due dates: plain `YYYY-MM-DD`. Public, unauthenticated, hostile-territory endpoints: `GET /pay/:token`, `POST /pay/:token/initiate`, `POST /webhooks/*`.

Build order — backend first, strictly

1 foundation (money, schema, **RLS + cross-tenant test**) → 2 identity/tenancy → 3 students → **4 billing** (hardest — over-test it) → **5 cash payments** (usable school, zero providers) → 6 mobile money + webhooks → 7 messaging → 8 analytics → 9 dashboard UI → 10 landing page.

UI

Dashboard: Next.js + TanStack Query + Tailwind + shadcn/ui + Recharts. **Mobile-first at 360 px — the director decides from a phone.** French-first, English toggle. **Optimistic UI is banned on money operations.** Landing: FintechX Framer look — `#1d1d1d` on `#edf1f4`, slate `#4d585f`, blue `#3b82f6`, emerald `#10b981`, Inter, scroll reveals, sticky step stacks. Calm and financial, not edtech.

Pricing (drives plan limits in code)

Essentiel 25 000 XAF/mo ≤250 students, 500 msgs · Croissance 60 000 XAF/mo ≤800, 2 000 msgs · Institution from 120 000 XAF/mo. Annual –20%. Prepaid message wallet (WhatsApp 10 XAF, SMS 30 XAF). Onboarding 150 000 XAF. Aggregator fees passed through, never absorbed.

The five things most likely to break

1. Balance projections drifting from the ledger → nightly integrity sweep, page on mismatch.
2. Tenant leaking through a pooled connection → `set local` in a transaction + a CI test.
3. A reminder storm or reminding someone who already paid → all caps in the sender.
4. Double payment on a bad network → idempotency keys + duplicate-settlement detection.
5. Receipt-number gaps read as deleted receipts by an auditor → locked counter row.

Complete Technical Architecture & Specifications

ARCHITECTURE — Fineduc

The complete technical file: stack, system design, data models, module boundaries, security, integrations, jobs, and deployment. This is the long one — it holds all the detail. For the fast reference an agent should load first, see [ARCHITECTURE-ESSENTIALS.md](#). For what and why, see [PRD.md](#).

- **Status:** v1 design, pre-code
- **Last updated:** 2026-08-18

1. Guiding principles

1. **The ledger is the product.** Everything else — reminders, dashboards, links — is a view over, or a trigger on, a correct student ledger. Get the ledger right and the rest is easy.
2. **Money is append-only.** No money row is ever updated or deleted. Corrections are new, signed, reason-coded rows. A balance is a projection, never an authoritative field you edit.
3. **Integers only, currency exponent respected.** XAF/XOF have **zero** decimal places. Store `bigint` in the currency's minor unit and carry the exponent explicitly. Never a float. Never a hard-coded `x 100`.
4. **Idempotency everywhere a write moves money or sends a message.** The network is bad, the webhooks retry, the cashier double-taps.
5. **The tenant boundary is enforced by the database, not by remembering a `where` clause.** Postgres Row-Level Security is the backstop for every application bug we have not written yet.
6. **One deployable, clean modules.** A modular monolith. Module boundaries are enforced by lint, not by network hops.
7. **External providers are ports.** Payment and messaging are interfaces with swappable adapters. No provider name appears in domain code, ever.
8. **Backend-first.** The API and the domain are complete and tested before a dashboard pixel is drawn. Every UI is a client of a documented API.

2. Stack

Layer	Choice	Why
Language	TypeScript 5.x , strict, <code>noUncheckedIndexedAccess</code>	One language across API, jobs, dashboard, landing. Types are the spec agents read.
Runtime	Node.js 22 LTS	Stable, cheap to host, huge ecosystem for the providers we need.
API framework	NestJS 11	Module boundaries, DI, guards and interceptors map exactly onto tenancy/auth/audit cross-cuts. Opinionated structure is a feature when agents write the code.
Database	PostgreSQL 16	Transactions, RLS, <code>numeric</code> / <code>bigint</code> , partial indexes, <code>jsonb</code> , generated columns. The tenancy and money model depend on it.
ORM / access	Prisma 6 for schema + typed reads, raw SQL for money-critical writes and reports	Prisma for velocity and a readable schema; SQL where locking and correctness matter.
Migrations	Prisma Migrate , forward-only, reviewed	
Cache / queue backing	Redis 7	Job queue, rate limits, idempotency keys, short-lived locks.
Jobs & scheduling	BullMQ	Reminder scheduling, webhook processing, reconciliation, exports. Retries with backoff, dead-letter queues, per-queue concurrency.
Validation	Zod at every boundary	One schema generates runtime validation + TS types + OpenAPI.
Auth	JWT access (15 min) + rotating refresh (30 d) , argon2id passwords, TOTP 2FA	
API docs	OpenAPI 3.1 , generated from Zod	The dashboard and landing consume it.
Dashboard	Next.js 15 (App Router) + TanStack Query + Tailwind + shadcn/ui + Recharts	Mobile-first; the director uses a phone.
Landing	Next.js 15 , static, + Framer Motion	Framer-template look; SSG for speed on 3G.
Files	S3-compatible object storage (Cloudflare R2)	Receipts, exports, student photos. Never in the database.
PDF	@react-pdf/renderer in a worker	Receipts and reports.
Email (staff only)	Resend	Guardians are reached by WhatsApp/SMS, never email.

Layer	Choice	Why
Errors / tracing	Sentry + OpenTelemetry → Grafana Cloud	
Logs	Pino , JSON, structured, <code>tenant_id</code> on every line, PII redacted	
Tests	Vitest (unit), Supertest (API), Testcontainers (real Postgres), Playwright (dashboard)	
Repo	pnpm workspaces + Turborepo monorepo	
CI	GitHub Actions	typecheck → lint → test → migrate-check → build
Hosting	API + workers on Railway or Fly.io (Paris/Frankfurt region), Postgres managed with PITR, web on Vercel	Low latency to West/Central Africa; cheap at our scale.

Deliberate non-choices

- **No microservices.** One API deployable, one worker deployable. Modules, not services.
- **No GraphQL.** REST + OpenAPI. Fewer moving parts, easier caching, easier for agents.
- **No event-sourcing framework.** The money tables *are* append-only; that is enough.
- **No Kafka.** BullMQ on Redis handles our volume by three orders of magnitude.
- **No direct telco integration.** One aggregator, behind a port. Non-negotiable — a direct MoMo/Orange contract drags us into licensing we do not want.

3. System shape



```

    ▲
    | signed webhooks (public endpoint,
    | signature-verified, replay-protected)
    aggregator callbacks

```

Monorepo layout

```

fineduc/
├─ apps/
│  ├─ api/           NestJS HTTP API           ← build first
│  ├─ worker/        BullMQ processors          ← build first
│  ├─ dashboard/     Next.js admin & director UI ← build second
│  ├─ pay/           Next.js public payment page ← build second
│  └─ web/           Next.js marketing landing   ← build last
├─ packages/
│  ├─ domain/        pure business logic, zero I/O ← the heart
│  ├─ db/            Prisma schema, migrations, seed
│  ├─ contracts/     Zod schemas + generated OpenAPI types (shared by api & clients)
│  ├─ providers/     payment + messaging ports and adapters
│  ├─ money/         Money value object, currency table, formatting
│  ├─ config/        typed env loading (Zod), fail-fast at boot
│  └─ ui/            shared React components + design tokens
├─ docs/            ADRs, runbooks, provider notes
└─ infra/           Docker Compose (dev), deployment config

```

Dependency rule, enforced by `eslint-plugin-boundaries` in CI: `domain` depends on nothing. `db`, `providers` may depend on `domain`, `money`, and `config`. `ui` and the client apps may depend on `money` for **display only** — formatting lives in `packages/money` and is never reimplemented elsewhere (AGENTS.md rule #1). `api` / `worker` depend on all packages. Apps never import from each other. **Nothing** imports from `apps/*`.

4. Multi-tenancy

Model: shared database, shared schema, `tenant_id` on every business table, isolated by **Postgres Row-Level Security**.

Why not a schema or database per tenant: a few hundred schools, and cross-tenant analytics, migrations and support all become painful. RLS gives us the isolation without the operational cost.

How it works

1. Every business table carries `tenant_id uuid not null`.
2. Every business table has `enable row level security` plus a policy: `using (tenant_id = current_setting('app.tenant_id')::uuid)`.
3. The API runs as a role **without** `BYPASSRLS`. Migrations run as a separate role that has it.

4. A `TenantContext` interceptor resolves the tenant from the JWT, opens a transaction, and issues `set local app.tenant_id = $1` before any query in the request.
5. `set local` is transaction-scoped, so a pooled connection can never leak a tenant into the next request.
6. Jobs set the same variable from the job payload's `tenantId`. A job without a tenant is rejected at enqueue time.

Test that must exist and must never be deleted: create two tenants with data, authenticate as tenant A, attempt to read every table, and assert zero tenant-B rows. Run it in CI on every PR.

Sites (campuses). A tenant has one or more `sites`. Students, cash desks and users belong to a site. Site is a *filter*, not an isolation boundary — a group director sees all sites, a site bursar is scoped to one.

5. Money

`packages/money` owns this and nothing else may reimplement it.

```
type Money = { amount: bigint; currency: CurrencyCode } // amount is in MINOR units
```

- Currencies carry an explicit exponent: **XAF 0, XOF 0**, NGN 2, GHS 2, USD 2, EUR 2. For XAF, one minor unit is one franc. There is no $\times 100$.
- Stored as two columns: `amount_minor bigint` + `currency char(3)`. Never `numeric`, never `float`, never a single "amount" column whose unit you have to guess.
- All arithmetic goes through the `Money` type. Adding two different currencies throws.
- **Allocation and split use largest-remainder** so the parts always re-sum to the whole. A three-way split of 100 000 XAF is 33 334 / 33 333 / 33 333 — never 33 333.33.
- Percentage discounts round **half-up to the minor unit**, computed on the base amount, and the resulting integer is what is stored — never the percentage re-applied later.
- Formatting is presentation-only, locale-aware (`fr-CM : 45 000 FCFA`), and lives in `ui`.
- A tenant has **one** currency, set at creation and immutable. A payment arriving in another currency is rejected and flagged for manual handling, never auto-converted.

6. Data model

Core tables. `tenant_id`, `created_at`, `updated_at`, `created_by` are on everything and omitted below for readability.

Tenancy and identity

- **tenant** — school. `name, legal_name, country, currency, timezone, locale, plan, status, logo_url, settings jsonb`
- **site** — campus. `tenant_id, name, address, is_primary`
- **user** — staff. `email, phone, password_hash, name, status, totp_secret_encrypted, totp_enabled, last_login_at, failed_login_count, locked_until`
- **membership** — user $\leftrightarrow$ tenant $\leftrightarrow$ role, optionally scoped to a site. `user_id, tenant_id, site_id, role, status` — a user may serve several schools (a group's accountant). **Unique on (user_id, tenant_id).**

- **refresh_token** — `user_id, token_hash, family_id, expires_at, revoked_at, user_agent, ip` (rotation with reuse detection)
- **api_key** — `tenant_id, name, key_hash, scopes[], last_used_at, expires_at` (Institution plan)

Academic structure

- **academic_year** — `name ("2026-2027"), starts_on, ends_on, status (draft|active|closed)` . **Only one active per tenant** (partial unique index).
- **term** — `academic_year_id, name, starts_on, ends_on, sequence`
- **grade_level** — `name ("6ème", "CM2"), sequence, cycle` (the class *level*)
- **class_group** — `grade_level_id, academic_year_id, site_id, name ("6ème A"), capacity, head_teacher_name`

People

- **student** — `matricule (unique per tenant), first_name, last_name, sex, born_on, photo_url, status (enrolled|left|graduated|suspended), notes`
- **guardian** — `first_name, last_name, phone_e164 (indexed), phone_alt_e164, email, relationship, preferred_channel (whatsapp|sms), preferred_locale, whatsapp_opt_in, opt_out_at, verification_status, bounce_count, quarantined_at`
- **student_guardian** — `student_id, guardian_id, is_primary, pays_fees, share_percent` (a fee split between two separated parents is a real case)
- **enrollment** — `student_id, class_group_id, academic_year_id, enrolled_on, left_on, status, fee_schedule_id, carried_forward_balance_minor` **The row that creates money owed.** Unique on `(student_id, academic_year_id)` .

Fees and the ledger

- **fee_schedule** — `academic_year_id, grade_level_id, name, version, effective_from, status (draft|published|archived), total_minor` — **versioned, never mutated after publish**
- **fee_item** — `fee_schedule_id, code, label, category (tuition|registration|exam|canteen|transport|uniform|boarding|other), amount_minor, is_mandatory, is_recurring, sequence`
- **instalment_plan** — `fee_schedule_id, name, instalment_count`
- **instalment_template** — `instalment_plan_id, sequence, label ("1ère tranche"), due_offset_days | due_on, percent_bp | amount_minor`
- **invoice** — one per enrollment per academic year. `enrollment_id, number, issued_on, total_minor, discount_minor, net_minor, paid_minor, balance_minor, status` (*paid_minor / balance_minor* are ***maintained projections***, recomputed inside the same transaction as every ledger write and verified nightly — never edited by hand.)
- **invoice_line** — `invoice_id, fee_item_id, label, amount_minor, quantity`
- **instalment** — `invoice_id, sequence, label, due_on (DATE, tenant timezone), amount_minor, allocated_minor, status (pending|partial|paid|overdue|waived|cancelled)` **The unit reminders and payments hang off.**
- **discount** — `invoice_id | invoice_line_id, type (sibling|staff|merit|hardship|commercial), method (percent|fixed), value, amount_minor, reason, granted_by, approved_by, granted_at`

- **adjustment** — `invoice_id, type (credit|debit), amount_minor, reason_code, note, created_by, approved_by` — how a mid-year fee change or a correction is recorded
- **student_ledger_entry** — **append-only.** `student_id, invoice_id, instalment_id, entry_type (charge|payment|discount|adjustment|refund|reversal|carry_forward), amount_minor (signed), balance_after_minor, source_type, source_id, occurred_on, memo` *The audit-grade truth of a student's account. Nothing here is ever updated or deleted.*

Payments

- **payment** — `student_id, invoice_id, method (mobile_money|cash|bank_transfer|cheque| card|waiver), amount_minor, currency, status (pending|processing|succeeded|failed| cancelled|expired|refunded|partially_refunded), provider, provider_ref, provider_fee_minor, fee_borne_by (payer|school), payer_phone_e164, payer_name, idempotency_key (unique per tenant), cash_session_id, initiated_by, received_at, reconciled_at, raw_provider_payload jsonb`
- **payment_allocation** — `payment_id, instalment_id, amount_minor` — how one payment is spread across instalments. A partial payment produces one row; a large payment produces several.
- **payment_link** — `student_id, invoice_id, instalment_id (nullable → "pay anything"), token (unguessable, 32 bytes), suggested_amount_minor, min_amount_minor, expires_at, used_at, created_by`
- **receipt** — `payment_id, number (gapless per tenant per year), issued_at, pdf_url, sent_at, sent_channel` — **immutable**; a correction issues a credit note, not an edit
- **refund** — `payment_id, amount_minor, reason_code, note, status, requested_by, approved_by, provider_ref, processed_at`
- **provider_event** — every inbound webhook, stored raw before processing. `provider, event_id (unique), event_type, signature_valid, payload jsonb, received_at, processed_at, processing_error, attempts` — **the replay log; also our idempotency guard**

Cash control

- **cash_desk** — `site_id, name, is_active`
- **cash_session** — `cash_desk_id, cashier_user_id, opened_at, opening_float_minor, closed_at, declared_close_minor, expected_close_minor, variance_minor, variance_reason, status (open|closed|reconciled|flagged), closed_by, reconciled_by` **Partial unique index: one open session per desk.**
- **cash_movement** — `cash_session_id, type (payment|float_in|float_out|deposit_to_bank| correction), amount_minor (signed), reference, note, created_by` — append-only

Messaging

- **message_template** — `code, channel, locale, subject, body, variables[], whatsapp_template_name, whatsapp_template_status, is_active` — WhatsApp templates need Meta pre-approval, so the approval status lives here
- **reminder_rule** — `tenant_id, name, offset_days (signed; -7 = seven days before due), channel, template_code, escalation_level, is_active, applies_to (all|class|status)`
- **reminder_schedule** — materialised intent. `instalment_id, reminder_rule_id, guardian_id, scheduled_for (timestamp tz, computed in tenant tz), status (scheduled|sent|skipped| cancelled|failed), skip_reason, message_id` — **unique on (instalment_id, rule_id, guardian_id)**

- **message** — guardian_id, student_id, channel, provider, to_phone_e164, template_code, locale, body_rendered, status (queued|sent|delivered|read|failed|undeliverable), provider_message_id, error_code, cost_minor, sent_at, delivered_at, read_at
- **message_credit_ledger** — append-only wallet. tenant_id, entry_type (topup|debit| refund|adjustment), amount_minor, balance_after_minor, message_id, note

Platform

- **subscription** — tenant_id, plan, billing_period, student_cap, price_minor, current_period_start, current_period_end, status
- **audit_log** — **append-only, no delete grant.** tenant_id, actor_user_id, actor_role, action, entity_type, entity_id, before jsonb, after jsonb, ip, user_agent, request_id, occurred_at
- **outbox** — transactional outbox. aggregate_type, aggregate_id, event_type, payload jsonb, published_at, attempts — a domain event is written in the *same transaction* as its state change, then published to the queue by a poller. This is how we avoid "payment saved but receipt never sent".

Indexes that matter

```
create unique index on student (tenant_id, matricule);
create index      on guardian (tenant_id, phone_e164);
create index      on instalment (tenant_id, due_on, status) where status in ('pending','partial','overdue');
create index      on payment (tenant_id, received_at desc);
create unique index on payment (tenant_id, idempotency_key);
create unique index on provider_event (provider, event_id);
create unique index on cash_session (cash_desk_id) where status = 'open';
create unique index on academic_year (tenant_id) where status = 'active';
create index      on student_ledger_entry (tenant_id, student_id, occurred_on desc);
create index      on reminder_schedule (status, scheduled_for) where status = 'scheduled';
create index      on audit_log (tenant_id, entity_type, entity_id, occurred_at desc);
```

7. Modules

Each is a NestJS module with a public service interface. Cross-module calls go through that interface only — never into another module's repository.

Module	Owns	Key operations
identity	user, membership, refresh_token, api_key	login, refresh, 2FA enrol/verify, invite, role change, password reset
tenancy	tenant, site, subscription, settings	onboard school, manage sites, plan limits, feature flags
academics	academic_year, term, grade_level, class_group	open/close a year, promote a cohort
students	student, guardian, student_guardian, enrollment	enrol, re-enrol, transfer, withdraw, attach guardian, merge duplicates
billing	fee_schedule, fee_item, instalment_plan, invoice, instalment, discount, adjustment, student_ledger_entry	publish a schedule, generate an invoice + instalments on enrolment , apply a discount, post an adjustment, compute a balance
payments	payment, payment_allocation, payment_link, receipt, refund, provider_event	initiate mobile money, record cash, process a webhook , allocate, issue a receipt, refund, reconcile
cashbox	cash_desk, cash_session, cash_movement	open, take cash, close with a count, reconcile, daily report
messaging	message_template, reminder_rule, reminder_schedule, message, message_credit_ledger	schedule reminders, render, send, track delivery, meter credits, honour opt-out
analytics	read-only projections	director dashboard, arrears ageing, recovery rate, method mix, reminder effectiveness
audit	audit_log	write (via interceptor), query, export
platform	health, config, jobs admin, provider status	ops surface

8. Critical flows

8.1 Enrolment creates the money owed

```
POST /enrollments { studentId, classGroupId, academicYearId, feeScheduleId?, planId? }  
↓ single transaction  
1. validate: year is active, student not already enrolled that year, schedule is published  
2. create enrollment  
3. resolve fee_schedule (explicit, or the published one for that grade_level + year)  
4. create invoice + invoice_lines from fee_items  
5. apply automatic discounts (sibling rule) → discount rows, recompute net  
6. expand instalment_templates → instalment rows  
   due_on = a DATE resolved in the TENANT timezone  
   amounts split by largest-remainder so they re-sum to net exactly  
7. if carried_forward_balance > 0 → a `carry_forward` ledger entry + an extra instalment  
8. write student_ledger_entry (type=charge) per line  
9. write outbox event `enrollment.created`  
↓ commit  
worker: schedule reminders for every instalment; send a welcome message to the guardian
```

Invariant checked in the same transaction: `sum(instalment.amount) == invoice.net_minor`.

8.2 Mobile money payment — the aggregator path

Guardian taps the link in the WhatsApp reminder

- GET /pay/:token (public, no auth, rate-limited)
resolve student + balance + suggested amount. Token is single-purpose and expiring.
- POST /pay/:token/initiate { amountMinor, payerPhone, operator }
Idempotency-Key header REQUIRED (client-generated UUID, replayed on retry)
 1. re-validate amount against the LIVE balance (the parent may have paid meanwhile)
 2. create payment(status=pending, provider_ref=null, idempotency_key)
 3. call PaymentProvider.initiate() → checkout URL / USSD push
 4. store provider_ref
 5. return the redirect
- parent authorises on their phone (MoMo / Orange / Wave PIN prompt)

— the browser redirect is a HINT. It is never trusted. —

- POST /webhooks/payments/:provider (public, signature-verified)
 1. verify HMAC signature; reject and alert on failure
 2. INSERT INTO provider_event (unique on provider+event_id)
→ conflict means we already have it → 200 OK, stop. This is the idempotency guard.
 3. enqueue webhook-processor job, return 200 within ~200 ms
(never do business work inside the webhook request)

worker: webhook-processor

1. SELECT ... FOR UPDATE on the payment
2. state machine: pending → succeeded | failed | expired. Illegal transitions are logged and dropped, never applied.
3. on success, in ONE transaction:
 - payment.status = succeeded, received_at, provider_fee_minor
 - allocate across instalments (oldest due first, or the pinned instalment)
 - payment_allocation rows
 - instalment.allocated_minor, status
 - student_ledger_entry(type=payment, signed, balance_after)
 - invoice.paid_minor / balance_minor recomputed
 - overpayment becomes a credit adjustment
 - CANCEL every scheduled reminder for the now-settled instalments
 - outbox: payment.succeeded
4. commit
5. from the outbox: render the receipt PDF → S3 → send it to the guardian on the channel they used

8.3 Cash payment at the desk

```
Cashier must have an OPEN cash_session, else the endpoint 409s.
POST /payments/cash { studentId, amountMinor, instalmentId?, payerName }
  Idempotency-Key REQUIRED – the cashier will double-tap on a slow connection
  one transaction: payment(method=cash, status=succeeded, cash_session_id)
                    + allocation + ledger entries + cash_movement + reminder cancellation
→ receipt number assigned GAPLESSLY from a per-tenant-per-year sequence
  (a dedicated counter row locked FOR UPDATE – a Postgres sequence has gaps on rollback,
   and gapless receipt numbering is an audit requirement)
→ print/share the receipt; optionally send it to the guardian
```

8.4 Closing the desk — the anti-leak control

```
POST /cash-sessions/:id/close { declaredMinor, note }
  expected = opening_float +  $\Sigma$  cash_movement(session)
  variance = declared - expected
  variance != 0 → variance_reason is REQUIRED, session status = flagged, director notified
  variance == 0 → status = closed
  the session becomes immutable; any later correction is a NEW movement in a NEW session,
  reason-coded and audited
```

8.5 Reminders — schedule, then decide at send time

```
Nightly (per tenant, at 02:00 tenant time) – reminder-scheduler
  for each unpaid instalment in the horizon (due_on within -60 .. +90 days):
    for each active reminder_rule:
      for each guardian with pays_fees = true and not opted out and not quarantined:
        upsert reminder_schedule(instalment, rule, guardian)
          scheduled_for = due_on + offset_days, at the tenant's send hour, in TENANT TZ
        (unique constraint makes the whole job safely re-runnable)

Every 15 min – message-sender picks up rows where scheduled_for <= now and status = scheduled
AT SEND TIME, and only at send time, check:
  X instalment already paid or waived      → skip (reason: settled)  ← the #1 trust bug
  X guardian opted out / quarantined      → skip
  X outside tenant quiet hours (default 07:00–20:00 tenant tz) → defer
  X guardian already messaged N times today (default 2) → defer
  X tenant daily message cap reached      → defer + warn the bursar
  X message credit balance == 0           → skip + alert (never send on credit)
then:
  render the template (locale, variables, school signature)
  mint a payment_link token
  MessagingProvider.send() – WhatsApp first; on undeliverable, fall back to SMS
  debit message_credit_ledger in the SAME transaction as the message row
  store provider_message_id for delivery callbacks
```

Hard limits enforced in the sender, not the scheduler — a scheduler bug must not be able to bypass them.

8.6 Reconciliation — the safety net

```
Every 30 min – reconciler
  payments in (pending, processing) older than 15 min
    → PaymentProvider.getStatus(provider_ref)
    → apply the same state machine as the webhook processor (shared code path)
  payments pending > 2 h → expire, cancel, notify the bursar
  daily 03:00 – integrity sweep, per tenant:
    Σ payment_allocation(instalment)      == instalment.allocated_minor
    Σ instalment.amount                   == invoice.net_minor
    Σ ledger entries(student)              == invoice.balance_minor
    Σ cash_movement(closed session)       == session.expected_close_minor
    latest ledger balance_after            == computed running balance
  any mismatch → the tenant is flagged, an incident is opened, the director is NOT shown a
  number we cannot prove. Silence is worse than an honest "reconciling".
```

9. Provider ports

No provider name appears in `packages/domain` or in any module service. Ever.

```
// packages/providers/payment/port.ts
export interface PaymentProvider {
  readonly name: string
  readonly supportedMethods: PaymentMethod[]
  initiate(req: InitiatePaymentRequest): Promise<InitiatePaymentResult>
  getStatus(providerRef: string): Promise<ProviderPaymentStatus>
  verifyWebhook(rawBody: Buffer, headers: Record<string, string>): WebhookVerification
  parseWebhook(payload: unknown): NormalizedPaymentEvent
  refund(providerRef: string, amount: Money, reason: string): Promise<RefundResult>
}

// packages/providers/messaging/port.ts
export interface MessagingProvider {
  readonly name: string
  readonly channel: 'whatsapp' | 'sms'
  send(msg: OutboundMessage): Promise<SendResult> // must accept an idempotencyKey
  parseStatusWebhook(payload: unknown): DeliveryStatusEvent[]
  estimateCost(msg: OutboundMessage): Money
}
```

Adapters, v1

- Payment: **CinetPay** (primary — MTN, Orange, Moov, Wave, card across Cameroon, CIV, Senegal, Burkina, Mali, Togo, Benin), **Flutterwave** (secondary — anglophone reach), `ManualProvider` (cash/transfer, no network), `FakeProvider` (deterministic, for tests).
- Messaging: **WhatsApp Cloud API** (Meta, direct — cheapest and richest), an **SMS aggregator** (Africa's Talking / a local one per country), `ConsoleProvider` (dev), `FakeProvider` (tests).

Adapter rules

- Every adapter is fully exercised by contract tests written against the *port*, so a new provider is proven by making the same suite pass.
- Adapters never write to the database. They translate, call, and return normalised results.
- Every outbound call has a timeout (10 s), a retry policy (3, exponential + jitter, only on 5xx/network), and a circuit breaker.
- Provider credentials come from env only, are never logged, and are validated at boot.
- **Webhook endpoints are public and therefore hostile territory**: verify the signature before parsing, use a raw-body parser, enforce a timestamp window against replay, rate-limit by IP, and store the raw event before doing anything with it.

10. Security

Tenant isolation — Postgres RLS as described in §4. The application `where` clause is a convenience; the policy is the guarantee. The API database role must not have `BYPASSRLS`.

Authentication — argon2id (memory 64 MB, iterations 3). Access JWT 15 min, refresh 30 d, rotated on every use with **reuse detection**: a replayed refresh token revokes the whole token family and alerts. TOTP 2FA mandatory for Director and Bursar, optional elsewhere. Account lockout after 5 failures, exponential backoff, unlock by the director or by email.

Authorisation — RBAC + explicit permission checks, deny by default.

Capability	Director	Bursar	Cashier	Secretary	Auditor
View dashboard & reports	✓	✓	✗	✗	✓ (read)
Enrol / edit students	✓	✓	✗	✓	✗
Publish a fee schedule	✓	✓	✗	✗	✗
Grant a discount / scholarship	✓	⚠ needs approval	✗	✗	✗
Take a cash payment	✓	✓	✓	✗	✗
Close a cash session	✓	✓	✓ (own only)	✗	✗
Reconcile a flagged session	✓	✗	✗	✗	✗
Refund / reverse a payment	⚠ dual approval	⚠ requests	✗	✗	✗
Send a manual reminder	✓	✓	✗	✓	✗
Change reminder rules / templates	✓	✓	✗	✗	✗
Manage users & roles	✓	✗	✗	✗	✗
Export all data	✓	✓	✗	✗	✓
View the audit log	✓	✓ (own tenant)	✗	✗	✓

Four-eyes controls — refunds, reversals, discounts above a tenant-set threshold, and cash variance reconciliation all require a second, different user. Enforced in the service layer with an `approval` record, not in the UI.

Audit — an interceptor writes an `audit_log` row for every mutating request: actor, role, action, entity, before/after diff, IP, user agent, request id. The table has `INSERT` and `SELECT` grants only; no `UPDATE`, no `DELETE`, for any application role. Retention 7 years.

Data protection — TLS 1.3 everywhere, HSTS. At rest: managed Postgres encryption plus application-level AES-256-GCM on TOTP secrets and provider credentials. Student photos in private S3 buckets reached only by short-lived signed URLs. PII (phone, name, matricule) redacted from logs by a Pino serialiser — enforced by a test that greps a log fixture for a known phone number.

Input & transport — Zod validation at every boundary, `helmet`, a strict CORS allowlist, a 1 MB body cap, `express-rate-limit` on Redis (auth 5/min/IP, payment initiation 10/min/token, public payment page 60/min/IP), and parameterised queries only.

Payment link tokens — 32 random bytes, base64url, single student, expiring (default 30 days), revoked on payment, rate-limited, and carrying **no** PII in the URL. The page shows a first name and a masked matricule, never a full identity.

Secrets — env only, loaded and validated by `packages/config` at boot, fail-fast if a required var is missing. `.env` is git-ignored; `.env.example` is committed with placeholders. No secret ever reaches a log, an error message, or a Sentry payload.

Compliance posture — Fineduc is **not** a payment institution: funds settle from the aggregator directly to the school's own account and never touch a Fineduc account. GDPR-shaped data handling as the baseline (it is stricter than most local regimes and travels well): purpose limitation, a documented retention schedule, export and erasure on request, and a DPA with the school. Guardians' phone numbers are processed on the school's legitimate interest in collecting fees, and opt-out is always honoured for marketing-shaped messages while transactional receipts remain.

11. Background jobs

Queue	Trigger	Concurrency	Retries	Notes
<code>webhook-processor</code>	on webhook receipt	10	5, exp backoff	Idempotent by <code>provider_event.event_id</code> . DLQ alerts immediately.
<code>reminder-scheduler</code>	cron, 02:00 tenant tz	1 per tenant	3	Idempotent via the unique constraint; safe to re-run any time.
<code>message-sender</code>	cron, every 15 min	5	3	All the safety checks live here.
<code>message-status</code>	on delivery webhook	10	3	Updates delivery/read state, drives fallback and quarantine.
<code>reconciler</code>	cron, every 30 min	1	3	Re-queries non-final payments. Shares the webhook state machine.
<code>integrity-sweep</code>	cron, 03:00 daily	1	1	The invariant checks in §8.6. Pages on mismatch.
<code>receipt-renderer</code>	outbox event	5	5	PDF → S3 → deliver.
<code>exporter</code>	on request	2	2	Large Excel/PDF exports, delivered as a signed URL.
<code>director-digest</code>	cron, 07:00 tenant tz	1 per tenant	2	The daily WhatsApp summary — the habit-forming feature.
<code>outbox-publisher</code>	every 5 s	1	∞	Polls the outbox, publishes to queues, marks published.

Every job payload carries `tenantId`, `requestId` and `attempt`. Every job sets `app.tenant_id` before touching the database. A job that cannot resolve a tenant fails loudly.

12. API surface

REST, `/api/v1`, JSON, Bearer JWT, OpenAPI-documented.

```

POST    /auth/login | /auth/refresh | /auth/logout | /auth/2fa/{enroll,verify}
GET     /me

GET     /students | POST /students | GET,PATCH /students/:id
GET     /students/:id/ledger           ← the student file: charges, payments, balance
GET     /students/:id/statement.pdf
POST    /students/:id/guardians | DELETE /students/:id/guardians/:gid

POST    /enrollments | POST /enrollments/:id/withdraw | POST /enrollments/bulk-promote

GET,POST /fee-schedules | POST /fee-schedules/:id/publish
GET,POST /instalment-plans

GET     /invoices/:id | GET /invoices/:id/instalments
POST    /invoices/:id/discounts | POST /invoices/:id/adjustments

POST    /payments/cash                 ← Idempotency-Key required
POST    /payments/manual               ← transfer / cheque
GET     /payments | GET /payments/:id
POST    /payments/:id/refund           ← dual approval
GET     /payments/:id/receipt.pdf | POST /payments/:id/receipt/send

POST    /payment-links | GET /payment-links/:id
GET     /pay/:token                   ← PUBLIC, no auth
POST    /pay/:token/initiate           ← PUBLIC, Idempotency-Key required
POST    /webhooks/payments/:provider   ← PUBLIC, signature-verified
POST    /webhooks/messages/:provider  ← PUBLIC, signature-verified

POST    /cash-sessions | POST /cash-sessions/:id/close | POST /cash-sessions/:id/reconcile
GET     /cash-sessions/:id/report.pdf

GET,POST,PATCH /reminder-rules
GET,POST,PATCH /message-templates

POST    /reminders/send                ← manual: one guardian, a class, or a debtor filter
GET     /messages                     ← the delivery log
GET     /message-credits | POST /message-credits/topup

GET     /dashboard/overview            ← the director screen, one call
GET     /dashboard/arrears?groupBy=class|age|amount
GET     /dashboard/collections?from&to&granularity
GET     /dashboard/cash-status
GET     /reports/:type/export?format=xlsx|pdf

GET     /audit-log

```

```
GET,PATCH /settings | GET,POST /users | PATCH /users/:id/role
GET /health | /health/ready | /health/providers
```

Conventions. Cursor pagination (`?cursor&limit` , max 100). Errors are RFC 9457 `application/problem+json` with a stable `type` , a `traceId` and, on validation failures, a field map. `Idempotency-Key` is mandatory on every money-moving POST and honoured for 24 h. Money crosses the wire as `{ "amountMinor": "45000", "currency": "XAF" }` — a **string** for the integer, so no JavaScript client can silently lose precision. Timestamps are ISO 8601 UTC; **due dates are plain** `YYYY-MM-DD` and must never be turned into a timestamp.

13. Dashboard

Next.js 15 App Router, server components for shell and static data, TanStack Query for live data, Tailwind + shadcn/ui, Recharts. Design tokens shared with the landing page via `packages/ui` .

Mobile-first is a hard requirement, not a nice-to-have. The director's primary device is a phone. Build every screen at 360 px first and let it grow.

- **Director home** — collected today vs expected, recovery-rate gauge, arrears by class, cash-desk status, method mix, a 30-day collection sparkline. One API call, cached 30 s, auto-refresh 60 s.
- **Students** — searchable list (name, matricule, phone), filters (class, status, balance), and the **student file**: identity, guardians, instalments with a status timeline, full ledger, message history, one-tap "send a reminder" and "record a payment".
- **Arrears** — the bursar's work queue. Filter, sort by amount or age, multi-select, bulk remind, export.
- **Cash desk** — open/close, live session total, movement list, close-with-count flow with the variance shown before confirmation.
- **Payments** — searchable, filterable, reconciliation status visible; a payment stuck in `pending` must be obvious and actionable.
- **Fees** — schedule builder, instalment-plan editor with a live preview of the dates and amounts a student will actually get.
- **Reminders** — rule ladder editor, template editor with a live preview and variable validation, delivery log, credit balance and burn rate.
- **Admin** — users and roles, sites, academic years, settings, audit log.

Accessibility and reality: WCAG 2.1 AA, works at 200% zoom, every state (loading, empty, error, offline) designed. Optimistic UI is **banned** on money operations — the cashier must see the server's truth, not a hopeful local one.

14. Environments and deployment

Env	Purpose	Data	Providers
local	Docker Compose: Postgres, Redis, MailHog	seeded fake school	Fake/Console adapters
staging	pre-production, full pipeline	anonymised	provider sandboxes
production	live schools	real	live credentials

CI (GitHub Actions): typecheck → lint (incl. boundary rules) → unit → integration (Testcontainers Postgres) → migration check (up, then down, then up again on a copy of the staging schema) → build. A PR cannot merge red.

CD: staging on merge to `main`, production on a tagged release with a manual gate. Migrations run as a separate step **before** the new code is released, and every migration must be backward-compatible with the currently-running version (expand → migrate → contract).

Backups: managed Postgres PITR, 30-day retention, plus a nightly logical dump to a different provider. **A restore is rehearsed monthly and the runbook lives in `docs/runbooks/`**. An unrehearsed backup is not a backup.

Observability: Sentry for errors, OpenTelemetry traces (request → job → provider call, one trace), Pino JSON logs with `tenant_id` and `request_id` on every line. Alert, with a human on the other end, on: webhook DLQ depth > 0 · reconciliation mismatch · message send failure rate > 5% · a tenant's message credits at zero · a cash session open

14 h · p95 latency > 1 s · any 5xx on a payment path.

15. Build order

Strictly backend-first. Nothing in a later phase starts before the phase above it is tested.

1. **Foundation** — monorepo, config, `money`, Prisma schema, migrations, RLS policies and the cross-tenant isolation test, seed script, health endpoints.
2. **Identity & tenancy** — auth, 2FA, RBAC, memberships, tenant context interceptor, audit interceptor.
3. **Academics & students** — years, classes, students, guardians, enrolment.
4. **Billing** — fee schedules, instalment plans, invoice + instalment generation, discounts, the student ledger.
This is the hardest and most important module. Over-test it.
5. **Payments — cash first.** Cash sessions, cash payments, allocation, receipts, closing and variance. A school is usable at this point with zero external providers.
6. **Payments — mobile money.** The `PaymentProvider` port, the CinetPay adapter, payment links, the public pay page, webhooks, the reconciler.
7. **Messaging.** The `MessagingProvider` port, templates, reminder rules, the scheduler, the sender with every safety check, the credit wallet, delivery tracking.
8. **Analytics.** Dashboard projections and exports.

9. **Dashboard UI.** Director home → students → arrears → cash → fees → reminders → admin.

10. **Landing page.** Framer-styled marketing site.

16. Hard questions

Technical answers to the questions in [PRD.md §9](#).

What will break?

- **Balance projections drifting from the ledger.** `invoice.paid_minor` is a cache; caches go stale. → It is only ever written inside the same transaction as the ledger entry, and the nightly integrity sweep recomputes and compares. A mismatch is an incident, not a warning.
- **Concurrent payments on the same instalment.** Two cashiers, or a cashier and a webhook, at the same moment. → `SELECT ... FOR UPDATE` on the invoice row before any allocation. All allocation flows through **one** service method. There is no second code path.
- **Webhook floods and replays.** → `provider_event` unique on `(provider, event_id)`; the endpoint stores raw and returns 200 in under ~200 ms; all work is queued.
- **Connection-pool tenant leakage.** The single worst possible bug in this system. → `set local` inside a transaction, never `set`; the API role lacks `BYPASSRLS`; an automated cross-tenant test runs on every PR.
- **Receipt-number gaps.** A Postgres sequence increments outside the transaction, so a rollback leaves a hole — and an auditor reads a hole as a deleted receipt. → A dedicated counter row per tenant per year, locked `FOR UPDATE`, incremented inside the transaction.
- **Timezone bugs on due dates.** → `due_on` is a `DATE`, resolved and compared in tenant time. A lint rule bans `new Date()` in `packages/domain`; a clock is injected.
- **bigint serialising to JSON.** `JSON.stringify(1n)` throws; `Number(bigint)` loses precision above 2^{53} . → Money crosses the wire as a string, enforced by the Zod contract.
- **A stuck queue nobody notices.** → DLQ depth is alerted, not merely dashboarded.
- **The aggregator changing its webhook payload without notice.** → Store raw, parse defensively, and treat a parse failure as a retryable job with a page, so an event is never silently dropped.
- **Prisma's connection pool versus RLS.** Prisma reuses connections aggressively. → Interactive transactions for every tenant-scoped request, and an integration test that hammers concurrent requests from two tenants and asserts zero bleed.

What edge cases are we missing?

Payment arriving after the academic year closed (accept it, post to the closed year with a reopen-audit entry, never silently drop) · guardian paying more than the total balance (overpayment → credit, surfaced, refundable) · a student withdrawing with a credit (refund flow, dual approval) · a fee schedule edited after invoices exist (blocked — publish a new version and post adjustments) · two guardians paying the same instalment simultaneously (row lock; the second becomes an overpayment/credit) · a webhook arriving for a payment we never created (log, alert, do **not** auto-create) · a guardian's phone number reassigned by the telco to a stranger (quarantine on bounces, re-verify) · a cashier deleted while holding an open session (block user deactivation while a session is open) · clock skew on the aggregator's timestamps (tolerate a window, do not reject) · a tenant changing timezone mid-year (freeze already-scheduled reminders, reschedule the rest) · daylight saving (none in our target countries, but the code must not assume it) · a leap-year due date on 29 February · an

instalment amount of zero (a full scholarship — must not generate a reminder) · a student in two schools of the same group (two tenants or two enrolments — decided as: two enrolments, one tenant, per-site).

What is over-engineered?

Cut, on purpose: **CQRS with separate read models** (Postgres views and good indexes are enough at a few hundred schools) · **an event-sourcing framework** (append-only money tables plus an outbox give us the audit trail without the machinery) · **Kubernetes** (two containers and a managed Postgres) · **a plugin architecture for providers** (two interfaces and a factory) · **a generic rules engine for reminders** (offset + channel + template covers every school we have met) · **GraphQL** (REST is fine and cacheable) · **a service mesh, feature-flag SaaS, or a data warehouse** (a boolean in `tenant.settings`; SQL on the primary; revisit at 100 schools) · **offline-first sync in the dashboard** (huge complexity, and a wrong offline balance is worse than a spinner — a small, explicitly-queued offline cash mode is v1.1 at the earliest) · **building our own PDF layout engine** · **supporting more than one currency per tenant.**

Development Principles & Invariants

(AGENTS.md)

AGENTS.md — how to work inside Fineduc

The single instruction file for any coding agent in this repo. `CLAUDE.md` points here. Do not duplicate rules across the two.

Read this first, in this order

1. [ARCHITECTURE-ESSENTIALS.md](#) — every task, always. It is short.
2. [PRD.md](#) — when the task touches product behaviour, pricing, or scope.
3. [ARCHITECTURE.md](#) — only the section you need. It is long; do not load it whole for a one-file change.

If the three disagree, **ARCHITECTURE.md wins on technical matters, PRD.md wins on product matters**. Then fix the disagreement in the same PR.

What this product is

Fee collection for private African schools. A file per student, dated instalments, automatic WhatsApp/SMS reminders, mobile money through a third-party aggregator, and a real-time view for the director. **It is not a school-management system** — no grades, no attendance, no timetable. If a request drifts there, say so before building it.

Non-negotiables

These are correctness rules, not style preferences. Violating one is a bug even if the tests pass.

1. **XAF and XOF have zero decimal places.** Money is `bigint` in minor units with an explicit currency exponent, and all arithmetic goes through `packages/money`. If you write a float, a `toFixed(2)`, or a hard-coded `* 100` for money, you have introduced a defect.
2. **Never update or delete a money row.** Payments, allocations and ledger entries are append-only. A correction is a new, signed, reason-coded row.
3. **Balances are projections.** Write them only inside the same transaction as the ledger entry that changes them. Never patch one to "fix" a number.
4. **Every tenant-scoped query runs inside a transaction that has issued** `set local app.tenant_id`. Never `set` (session-scoped — it leaks across pooled connections). Never add `BYPASSRLS` to the application role.

5. **Every money-moving POST and every message send requires an idempotency key.**
6. **The payment webhook is the source of truth.** Never mark a payment succeeded from a browser redirect or a client call.
7. **Reminder eligibility is re-checked at send time against the live balance.** Never send from a decision made at schedule time.
8. **No provider name in `packages/domain` or in any module service.** Go through the port.
9. `due_on` is a `DATE` in the tenant's timezone. Never a timestamp, never UTC arithmetic on a due date.
10. **Every mutating endpoint writes an audit row.** If you add a mutation and no audit row appears, the feature is incomplete.
11. **Never log a phone number, a full name, a matricule, a token, or a secret.**
12. **Fineduc never holds funds.** If a design implies money passing through our account, stop and flag it.

How to build a feature

Work in this order. Do not jump to the controller.

1. **Contract first** — a Zod schema in `packages/contracts`. Types are derived from it, never hand-written alongside it.
2. **Domain logic** — pure functions in `packages/domain`. No database, no HTTP, no clock, no randomness. Inject anything impure.
3. **Persistence** — Prisma schema change plus a migration. Money-critical writes use raw SQL with explicit locking.
4. **Service** — the module service orchestrates: transaction boundary, locks, ledger writes, outbox event.
5. **HTTP** — a thin NestJS controller. Validate, delegate, map the result. **No business logic in a controller.**
6. **Job** — if there is asynchronous work, it is triggered by an outbox event, not by a direct `queue.add()` inside the transaction.
7. **Tests** — see below.
8. **Docs** — if you changed a rule, a model or a flow, update ARCHITECTURE.md **and** the corresponding line in ARCHITECTURE-ESSENTIALS.md in the same PR.

Tooling gotcha: never run a NestJS app through `tsx`

`tsx` (and any esbuild-based runner) does not implement TypeScript's `emitDecoratorMetadata`. NestJS's constructor DI reads that metadata to know which provider to inject for `private readonly foo: FooService` — without it, Nest boots and logs as if everything worked, but silently injects `undefined` for every constructor parameter, only crashing later when the handler actually calls a method on it. `packages/db/src/seed.ts` (a plain script, no DI) can use `tsx` freely; `apps/api` and `apps/worker` cannot. Always run them via `tsc` + `node dist/...` — `pnpm dev` uses `tsc-watch` for exactly this reason. Don't "simplify" it back to `tsx watch`.

Testing

Write these, in this priority order:

- **Domain unit tests** — the money maths, allocation, largest-remainder splitting, instalment generation, discount application, the payment state machine. Fast, no I/O, exhaustive.
- **Integration tests** against a real Postgres via Testcontainers — RLS isolation, transaction boundaries, locking, the gapless receipt counter under concurrency.
- **Provider contract tests** written against the port. A new adapter is proven by making the existing suite pass. Never test against a live provider.
- **API tests** with Supertest — authorisation matrix, idempotency replay, validation errors.
- **E2E** with Playwright for exactly three journeys: enrol → invoice, cash payment → receipt → close the desk, reminder → payment link → webhook → settled.

Tests that must exist and must never be deleted:

- Two tenants, authenticate as A, attempt to read every table, assert zero B rows.
- A webhook delivered twice produces exactly one settlement.
- A reminder is not sent for an instalment paid between scheduling and sending.
- Concurrent payments on one instalment do not over-allocate.
- 100 000 XAF split three ways re-sums to exactly 100 000.
- A log fixture contains no phone number.

Money paths are not "covered enough" at 80%. Get them to 100% of branches.

Code conventions

- TypeScript strict, `noUncheckedIndexedAccess`, no `any`, no non-null `!` on anything that crosses a boundary. `unknown` + a Zod parse instead of a cast.
- Errors: typed domain errors in `domain`, mapped to RFC 9457 `problem+json` at the HTTP edge. Never throw a bare `Error` with a string for something a client must handle.
- Naming: `snake_case` in the database, `camelCase` in TypeScript, `kebab-case` for files. Money fields always end in `_minor` / `Minor`. Dates that are calendar dates end in `_on`; instants end in `_at`.
- No barrel `index.ts` re-export files except at a package root.
- Comments explain **why**, never what. Match the density of the surrounding code.
- Commits: conventional commits, imperative, scoped — `feat(billing): generate instalments on enrolment`.
- French is the product's first language, but **code, identifiers, comments, commit messages and these docs are in English**. Domain terms may carry the French in a comment (`// grille tarifaire`).

Things you must not do without asking

- Add a dependency that is not already in the lockfile.
- Add a new third-party service, a new provider, or a new hosting component.
- Change the money representation, the tenancy strategy, or the append-only rule.
- Run a destructive migration, or any migration that is not backward-compatible with the currently-deployed code.

- Touch production data, or run anything against production credentials.
- Weaken or bypass a rate limit, a quiet-hours check, or a message cap "to make a test pass".
- Send a real message or initiate a real payment from a dev or test environment. Use the Fake adapters.
- Widen v1 scope into grades, attendance, timetable, payroll, or a parent portal.

When something is ambiguous

Do the part that is unambiguous, then state the assumption or ask the one question that actually changes the work. Do not stop with nothing delivered, and do not silently pick a money-affecting default.

Before you say a task is done

- ☐ `pnpm typecheck && pnpm lint && pnpm test` all pass
- ☐ New money path has domain tests **and** an integration test
- ☐ Mutation writes an audit row
- ☐ Money-moving endpoint accepts and honours `Idempotency-Key`
- ☐ Tenant-scoped query runs under `set local app.tenant_id`
- ☐ No phone number, name, token or secret in any log line
- ☐ Migration is backward-compatible and reversible
- ☐ `.env.example` updated if a new variable exists
- ☐ Docs updated if a rule, model or flow changed
- ☐ You have said plainly what you did **not** do

Hard questions — ask these of your own work

Before opening a PR, answer these three out loud in the PR description. They are the reason the rest of this repo's documentation exists.

- **What will break?** Under concurrency, under a bad network, on a retry, at a tenant boundary, across a timezone, at the end of an academic year.
- **What edge cases am I missing?** Partial payment · overpayment · a refund on a partially allocated payment · a zero-amount instalment from a full scholarship · a payment for a closed year · a guardian with children in several classes · a wrong or reassigned phone number.
- **What is over-engineered?** If you added an abstraction with one implementation, a queue for work that is synchronous, a cache for a query that runs twice a day, or a configuration option no school has asked for — delete it before review.

Current state

Pre-code. The scaffold in `apps/` and `packages/` is intentionally empty or barely drafted — it exists to fix the shape of the project, not to be working software. Start at [ARCHITECTURE.md §15, Build order](#), phase 1. Do not start at phase 9 because the UI is more fun.